Intelligent Hybrid Book Recommendation System Using Machine Learning

A MINI PROJECT REPORT

Submitted by

Kshitij Varma (2400520100046)

Sachin Srivastava (2400520100062)

Samyak Jain (2400520100064)

Shashwat Gangwar (2400520100066)

Sumit Rawat (2400520100067)

in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Under the supervision of

Mr. Abhshek Nagar, Ms Kajal

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Institute of Engineering and Technology, Sitapur road Lucknow 226021

Dr. A P J Abdul Kalam Technical University, Lucknow

SESSION 2025-2026

CERTIFICATE

This is to certify that **Kshitij Varma, Sachin Srivastava, Samyak Jain, Shashwat Gangwar and Sumit Rawat**, students of the Computer Science and Engineering branch, have successfully completed the mini-project titled **"Book Recommendation System"** during the **4th semester** of the B.Tech. program for the academic year **2025–26**.

The project was carried out under the guidance of **Mr. Abhshek Nagar and Ms Kajal** during which the student demonstrated commendable diligence, dedication, and commitment.

This project was submitted in partial fulfillment of the requirements of the B.Tech. program and has been duly evaluated by the undersigned.

We extend our best wishes to the student for continued success in future endeavors.

Date: 20/04/2026

Signature of Supervisor
Mr. Abhshek Nagar
System Manager
CSED, IET LKO

Ms Kajal
Assistant Professor (CF)
CSED, IET LKO

ABSTRACT

The exponential growth of digital libraries has created an overwhelming abundance of literature, making it increasingly difficult for readers to discover books tailored to their individual preferences. To address this challenge of information overload, this project presents an Adaptive Hybrid Book Recommendation System. Engineered with Python, Flask, SQLite, and Scikit-Learn, the application utilizes a dual-engine architecture that combines a Popularity-Based baseline for new users with an advanced Item-Item Collaborative Filtering model to deliver highly personalized suggestions based on historical reading patterns and Cosine Similarity.

Unlike traditional static machine learning models, this system is designed for dynamic state management and continuous learning. It features a robust RESTful architecture that integrates the Google Books API, allowing users to actively expand the database with new titles and submit explicit numerical feedback. To maintain high web server performance without sacrificing accuracy, computationally expensive matrix recalculations are decoupled into an asynchronous batch-processing pipeline that serializes the updated mathematical models. Fully configured for persistent cloud deployment on an AWS EC2 virtual machine, this project successfully bridges the gap between static data science experiments and scalable, interactive web engineering, delivering a highly personalized and continuously evolving user experience.

TABLE OF CONTENTS

01.Introduction

1.1 Overview of the Project

1.2 Problem Statement

1.3 Scope of the Project

02.Workflow

2.1 Database & API Selection Phase

2.2 User Input Phase

2.3 Machine Learning Phase

2.4 Output Rendering & Interpretation

03.Data Flow Diagram

3.1 Data Flow Diagram

04.Methodology

4.1 Project Methodology Overview

4.2 Batch Retraining Strategy

4.3 Query Mapping and Vector Construction

05.Implementation

5.1 System Architecture

5.2 Technology Stack Used

5.3 Frontend Implementation

5.4 API Integration and Query Handling

5.5 Backend Implementation

06. Working Screenshots

07.Technical scope and Future enhancements

7.1 Technical Scope of the System

7.2 Limitations of the Current Approach

7.3 Potential Enhancements

08.Conclusion

09.References

INTRODUCTION

1.1 Overview of the Project

The Book Recommendation System is a web-based machine learning application designed to help users discover new literature based on their reading preferences. It utilizes a hybrid approach, offering both a Popularity-Based recommender for trending books and a Collaborative Filtering model to provide highly personalized suggestions. The system is dynamic, allowing users to actively contribute to the dataset by adding new books via external APIs and submitting custom ratings.

1.2 Problem Statement

With the exponential growth of digital libraries, users frequently face information overload when attempting to select their next read. Traditional static libraries lack personalization. Furthermore, recommendation systems often suffer from the "cold start" problem, where new items cannot be recommended until they gather sufficient data. This project addresses the need for an adaptive, user-driven system that mathematically correlates user preferences to surface relevant titles.

1.3 Scope of the Project

The current scope includes a responsive web interface where users can view the top 50 highest-rated books, search for specific titles to receive localized recommendations, dynamically add new books using 13-digit ISBNs, and rate books anonymously.

Workflow

2.1 Database & API Selection Phase

To ensure data persistence without the overhead of a dedicated database server during initial development, SQLite was selected for local storage via SQLAlchemy. To ensure clean and accurate metadata for new entries, the system integrates the Google Books API rather than relying on brittle web-scraping techniques.

2.2 User Input Phase

Users interact with the system through Bootstrap-styled web forms. They can submit a book title to receive recommendations, input an ISBN to fetch external data, or submit a numeric rating (1-10) alongside a user identifier.

2.3 Machine Learning Phase

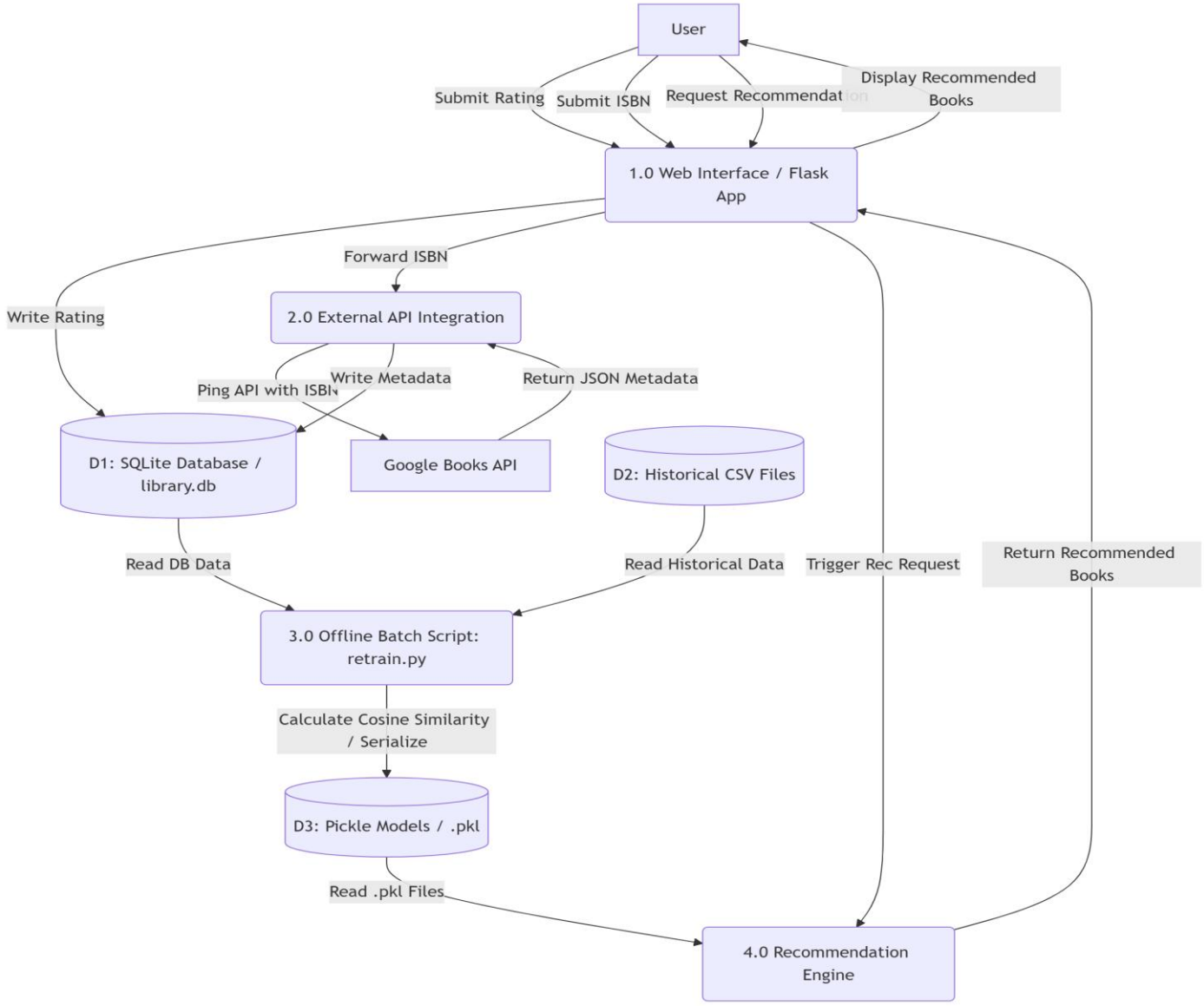
Upon receiving a recommendation request, the system references a pre-computed matrix. It utilizes Cosine Similarity to compare the vector of the requested book against the vectors of all other books in the database, identifying the closest mathematical matches based on historical user ratings.

2.4 Output Rendering & Interpretation

The Python backend processes the similarity scores, retrieves the corresponding metadata (title, author, cover image) from the database, and passes this array to Jinja2 templating engine. The HTML is dynamically rendered and returned to the client's browser.

Data Flow Diagram

The data flow initiates when a user submits a book rating or an ISBN through the Flask web interface. Ratings are written directly to a local SQLite database, whereas ISBN submissions trigger the backend to fetch and store JSON metadata from the external Google Books API. Asynchronously, an offline batch script (retrain.py) aggregates this live database with historical CSV files to compute a Cosine Similarity matrix, serializing the mathematical relationships into Pickle (.pkl) models. When a user subsequently requests a recommendation, the Flask backend queries these pre-computed models to rapidly retrieve the most relevant titles, formatting the data and delivering the final book suggestions back to the frontend interface.



Methodology

4.1 Project Methodology Overview

The system employs a dual-algorithm methodology. The homepage utilizes a Popularity-Based approach, mathematically sorting books by their average rating, filtered to only include books with a statistically significant number of votes (e.g., >250). The recommendation engine utilizes Item-Item Collaborative Filtering, analyzing user interaction patterns to group similar texts.

4.2 Batch Retraining Strategy

Computing distance matrices is computationally expensive. To prevent the web application from hanging during user interactions, the system utilizes a batch-processing methodology. Fresh data is stored safely in an SQLite database. A standalone pipeline script (retrain.py) merges this fresh data with the historical CSV baseline, recalculates the matrices, and serializes the updated models using Python's pickle library for rapid memory loading upon server restart.

4.3 Query Mapping and Vector Construction

The core collaborative filtering logic is built on a pivot table where rows represent books, columns represent users, and values represent ratings. To find similar books, the system calculates the cosine of the angle between two book vectors. The formula used is:

$$similarity = cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Where A and B represent the rating vectors of two different books. A score closer to 1 indicates high similarity.

5

IMPLEMENTATION

5.1 System Architecture

The project follows a classic Model-View-Controller (MVC) architectural pattern, adapted for Python. It relies on a persistent file system to manage the serialized machine learning models, making it highly compatible with persistent cloud virtual machines.

5.2 Technology Stack Used

- **Backend:** Python 3.x, Flask, Gunicorn
- **Machine Learning:** NumPy, Pandas, Scikit-Learn
- **Database:** SQLite, SQLAlchemy ORM
- **Frontend:** HTML5, CSS3, Jinja2, Bootstrap 5
- **External Services:** Google Books API

5.3 Frontend Implementation

The user interface is built using Jinja2 templates extending standard HTML. Bootstrap 5 is utilized for mobile responsiveness and grid layouts. Custom CSS is employed to create a modern, dark-mode aesthetic with interactive hover states for the book cards, ensuring high scannability.

5.4 API Integration and Query Handling

The requests library is used to handle HTTP GET requests to the Google Books API (https://www.googleapis.com/books/v1/volumes?q=isbn:{isbn}). The JSON payload is parsed to extract the title, concatenated author strings, and thumbnail URLs, handling missing keys gracefully to prevent server crashes.

5.5 Backend Implementation

Flask serves as the routing engine. It initializes the database context, loads the .pkl models into memory during startup, and exposes RESTful endpoints (/, /recommend, /add_book,

/rate_book). Error handling is implemented to catch IndexError exceptions if a user searches for a book not yet present in the mathematical matrix.

WORKING SCREENSHOTS

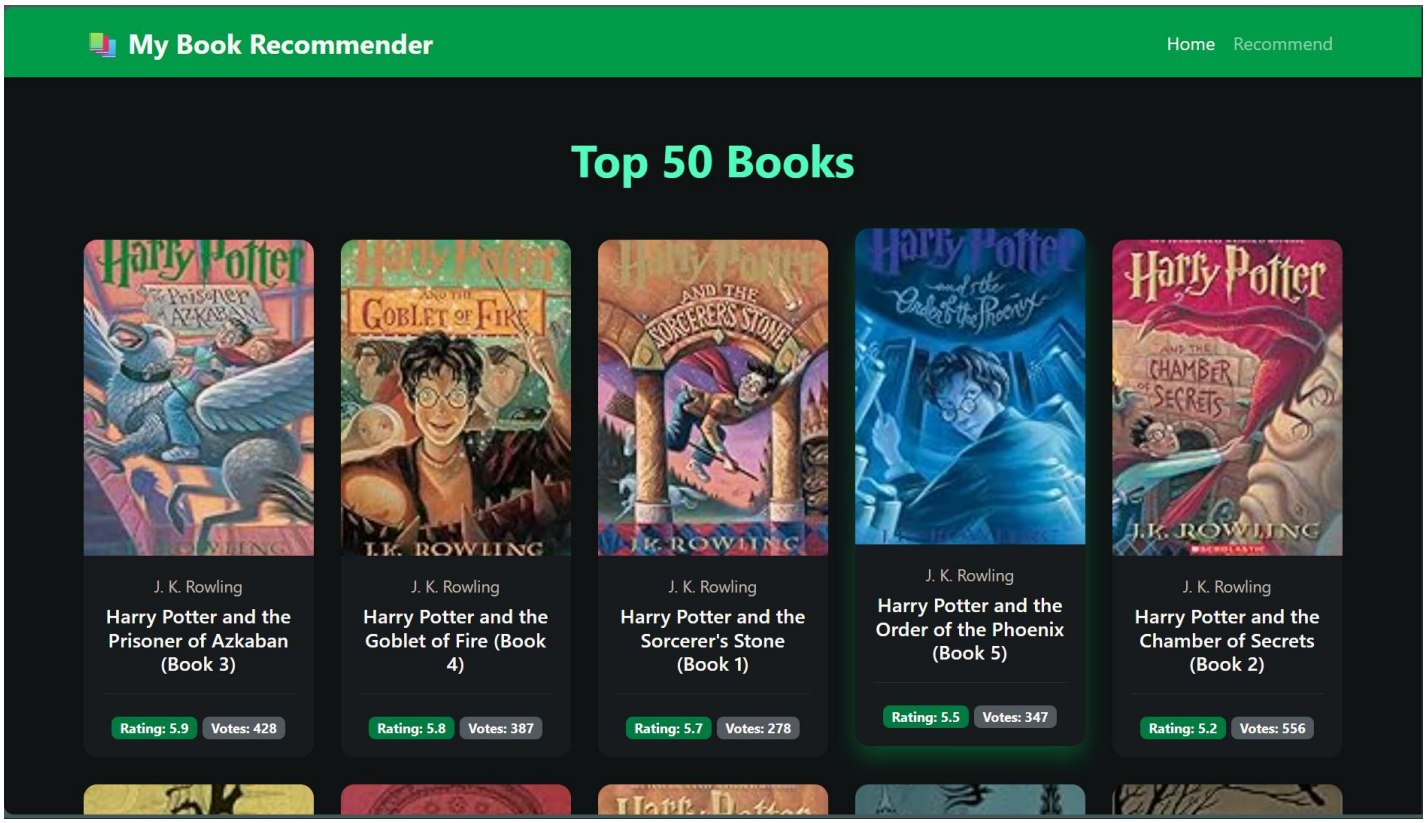


Fig 6.1 Home page showing Top 50 books

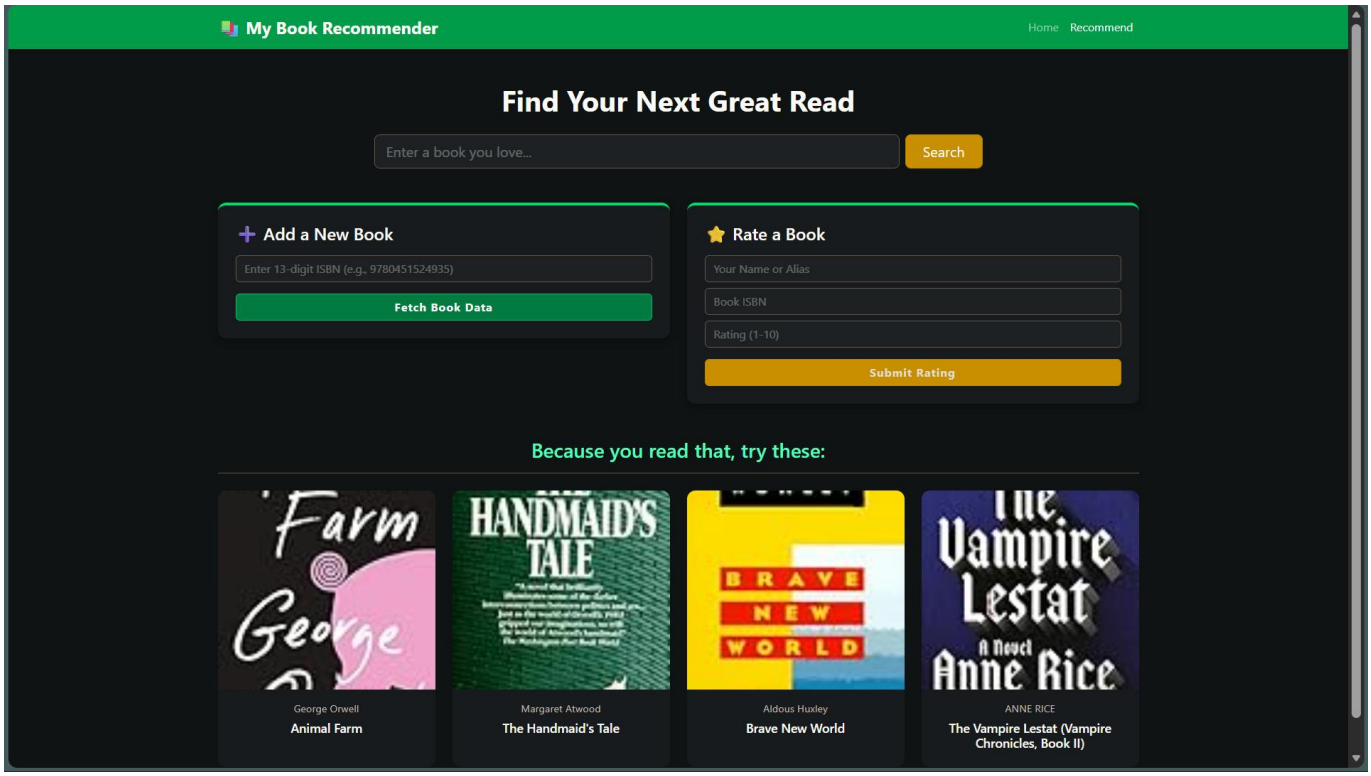


Fig 6.2 Recommendation based on user input

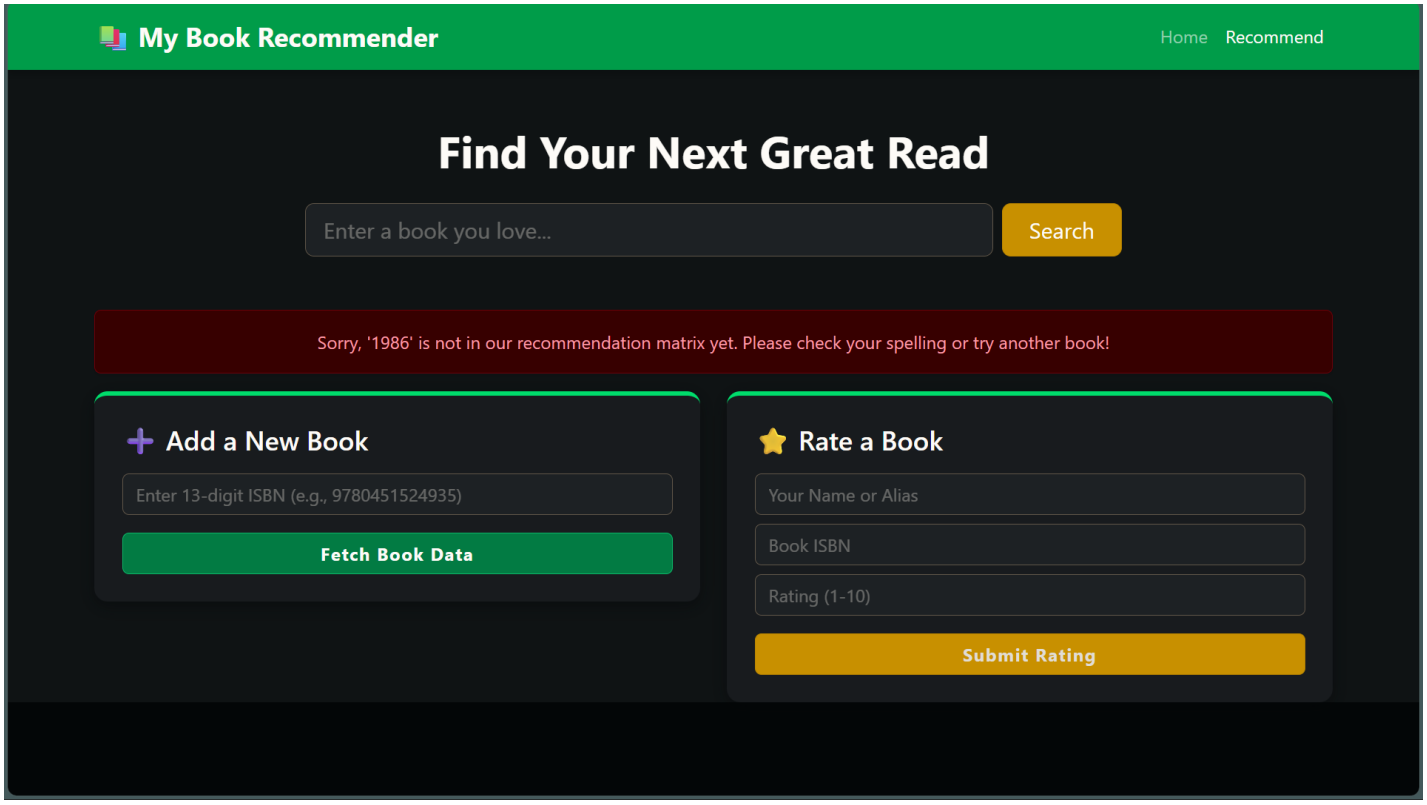


Fig 6.3 Error message when entering wrong book name

TECHNICAL SCOPE AND FUTURE ENHANCEMENTS

7.1 Technical Scope of the System

The system currently processes explicit feedback (direct 1-10 numerical ratings) to map user preferences. It successfully handles data cleaning, memory management for large sparse matrices, and external REST API consumption.

7.2 Limitations of the Current Approach

- **Sparsity:** Because the matrix relies on exact user overlap, highly obscure books may struggle to generate accurate recommendations.
- **Strict Thresholds:** Books require a minimum of 50 ratings, and users a minimum of 200 ratings, to be considered statistically significant, leading to a delay before new data influences the algorithm.

7.3 Potential Enhancements

- **User Authentication:** Implementing hashed passwords and user sessions to track individual reading histories.
- **Hybrid Recommendation Engine:** Incorporating Natural Language Processing (NLP) to analyze book descriptions, allowing the system to recommend new books based on content/genre before they reach the 50-rating threshold.
- **Database Migration:** Upgrading from SQLite to a robust cloud database like PostgreSQL to handle high-concurrency transactions.

CONCLUSION

The Book Recommendation System successfully bridges the gap between static data science scripts and interactive web engineering. By implementing an adaptive architecture that handles external API data, database state management, and batch machine learning retraining, the project effectively solves the problem of discovering personalized literature in a scalable manner.

REFERENCES

[1] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1–19:19.

[2] Streamlit Official Documentation, Snowflake Inc., 2024.

[3] Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.

[4] "Book-Crossing Dataset." Kaggle, collected by Cai-Nicolas Ziegler.

[5] Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based Recommender Systems: State of the Art and Trends. Recommender Systems Handbook, 73-105.